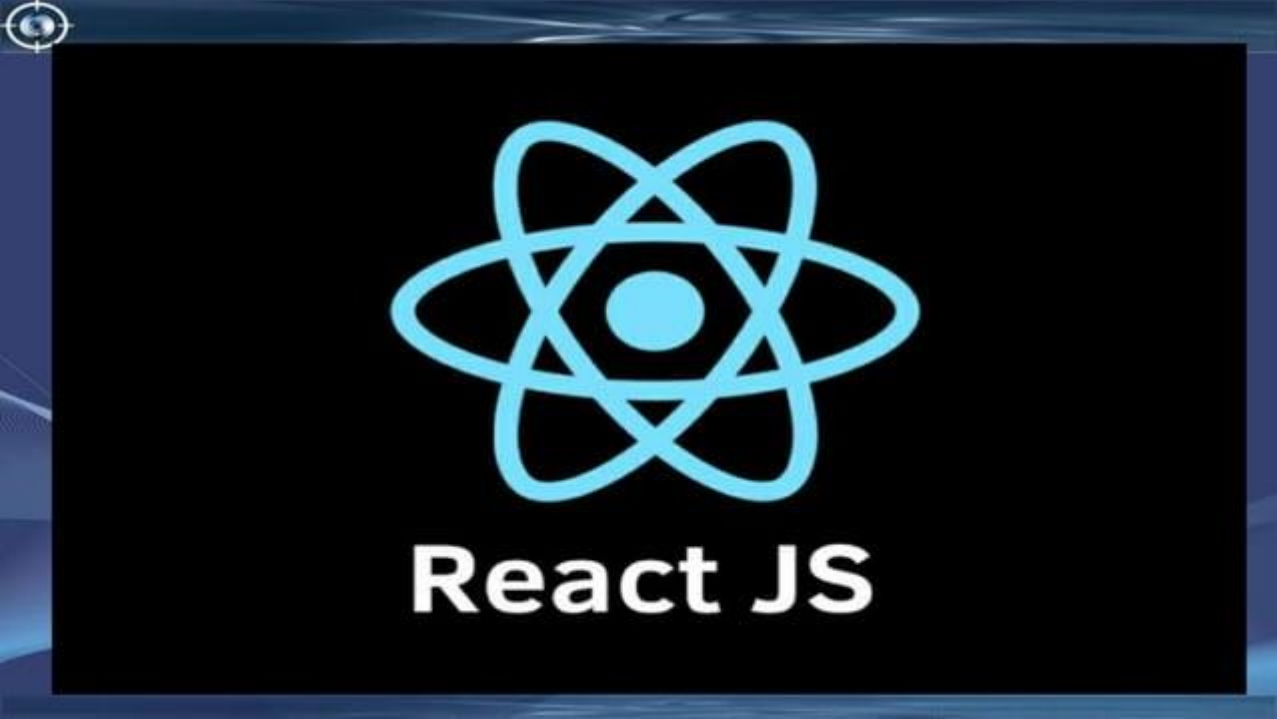

React JS

CONTENTS:

- Introduction and Overview
- Pre-requistes
- Installation
- Fundamentals of React JS
- React Components and Life cycle
- React Router
- React Hooks
- React Redux
- React Projects
- Testing in Reactjs
- React v/s Angular
- Tips to React Developer

WHAT IS REACTJS

- React is a free and open source java script library for building a composable user interfaces.
- It supports to build reusable user interface components which is changes over time.
- It abstracts as simple programming language and better performance model
- It can be base in the development of single-page or mobile applications
- Rect adheres to the declarative programming paradigm.
- Developers: FACEBOOK
- Developer(s) : Jordan Walke, Meta and Community

- Notable features .
  - Declarative UI
  - Components
  - Functional Components
  - Class-based Components
  - Virtual DOM
  - Life cycle methods
  - Javascript XML or JSX
  - React hooks
  - Rect Native
  - One-way Data binding

Advantages

- Easy to learn
- Enabes Building Rich UI
- Facilitates creation of custom components
- It Boosts Developer Productivity
- Quick Rendering
- SEO-Friendly
- Helpful developer Toolset
- Enhanced Code stability
- Large Community support
- High Demand due to fast speed

Disadvantages

- Lack of adequate documentation
- Only covers user interface
- High pace
- Migrating data from Rectjs
- Setting up React

When to user Reactjs?

- User friendly, fast and responsive single page application
- Creating interactive elements within UI
- Develop components for high-volume, high-performance apps
- Develop quick loading webpps
- Creative mobile apps in native platform
- Ensuring stable and steady coding
- Creating apps that are SEO Friendly
- Making scalable apps when you may need to frequently add new features

Pre-requistes?

- Javascript
  - DOM- How to render dom?
  - Core concepts – hoisting, Call Apply, Bind, Higher order function,closure and object oriented javascripts
- ECMASCRIPT(ES6)
- JSX(xml version of HTML)
- OOPS concepts
- Basic CSS Stuffs like Box Model, Position etc
- Webapps
  - Ajax calls
  - Single Page Applications
  - Responsive Web design

- Developers: Facebook
- React was originally created by **Jordan Walke.**
- Current version of React.JS is V17.0.2 (August 2021).
- Initial Release to the Public (V0.3.0) was in July 2013.
- React.JS was first used in 2011 for Facebook's Newsfeed feature.
- Current version of create-react-app is v4.0.3(August 2021).
- create-react-app includes built tools such as webpack(bundle tool), Babel(JavaScript transcompiler), and ESLint(Static analyzer).
- Apps with React is : Netflix, Whatsapp Web, Instagram, Airbnb

Essential ECMAScript 2015 / ES6

- ES6 is the **newer standardization/version of Javascript**, which was released in 2015. It is important to learn ES6, because it has many new features that help developers write and understand JavaScript more easily.

- NEW FEATURES OF ES6

- The let keyword
- The const keyword
- Arrow Functions
- For/of
- Map Objects
- Set Objects
- Classes
- Promises
- Symbol
- Default Parameters
- Function Rest Parameter
- String.includes()
- String.startsWith()
- String.endsWith()
- Array.from()
- Array keys()
- Array find()
- Array findIndex()
- New Math Methods
- New Number Properties
- New Number Methods
- New Global Methods
- Object entries
- JavaScript Modules

- https://reactjs.org/docs/cdn-links.html
- https://cdnjs.com/libraries/babel-standalone

```
<script src="https://unpkg.com/react@17/umd/react.production.min.js"></script>
<script src="https://unpkg.com/react-dom@17/umd/react-dom.production.min.js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/babel-
standalone/6.26.0/babel.min.js"></script>
```

INTRODUCTION TO NODE.JS

- Node.js is an open source server environment
- Node.js is free
- Node.js runs on various platforms (Windows, Linux, Unix, Mac OS X, etc.)
- Node.js uses JavaScript on the server
- Node.js files contain tasks that will be executed on certain events
- A typical event is someone trying to access a port on the server
- Node.js files must be initiated on the server before having any effect
- Node.js files have extension ".js"

Setting up Development Environment

- **Step 1**: Install NodeJS. You may visit the official download link of NodeJS to download and install the latest version of NodeJS. Once we have set up NodeJS on our PC, the next thing we need to do is set up React Boilerplate.

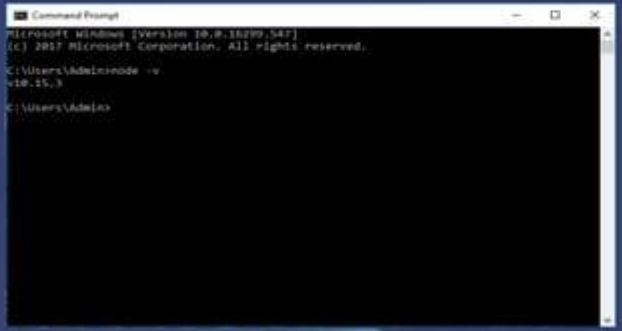
- **Step 2**: Setting up react environment for older and latest versions, follow anyone according to your node version.

- **For Older Versions which include Node < 8.10 and npm < 5.6**: Setting up React Boilerplate. We will install the boilerplate globally. Run the below command in your terminal or command prompt to install the React Boilerplate.

- Installation of Node.js on Windows
- Setting up the Node Development Environment
- **Step-1:** Downloading the Node.js '.msi' installer.

- **Step-2:** Running the Node.js installer.

- **Step 3: Verify that Node.js was properly installed or not.**



- **Step 4: Updating the Local npm version.**

The final step in node.js installed is the updation of your local npm version(if required) – the package manager that comes bundled with Node.js.

You can run the following command, to quickly update the npm

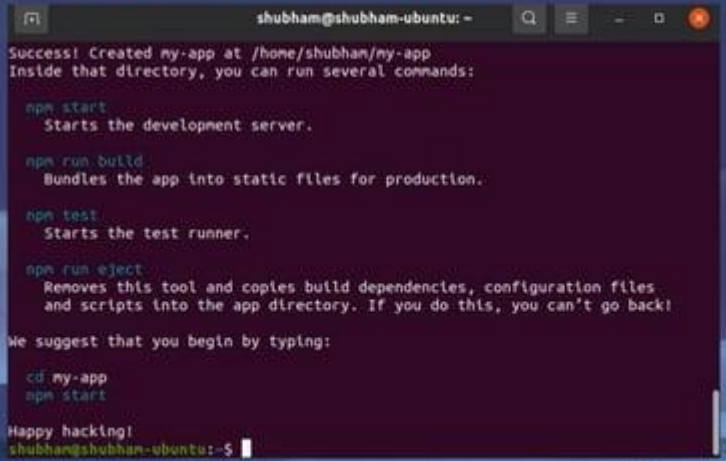
npm install npm –global // Updates the 'CLI' client

npm install -g create-react-app

- After running the above command and successfully installing the boilerplate your terminal will show some output as shown in the below image:

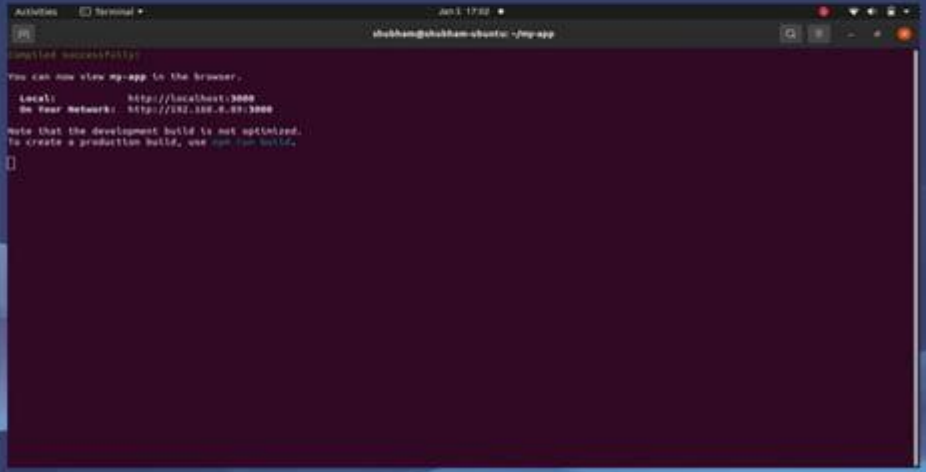
- **For the Latest Versions which include Node >=8.10 and npm >=5.6:** For using the latest features of JavaScript which provides a good development experience the machine should contain a version of Node >=8.10 and npm >=5.6.
- Run the below command to create a new project.

```
npx create-react-app my-app
```

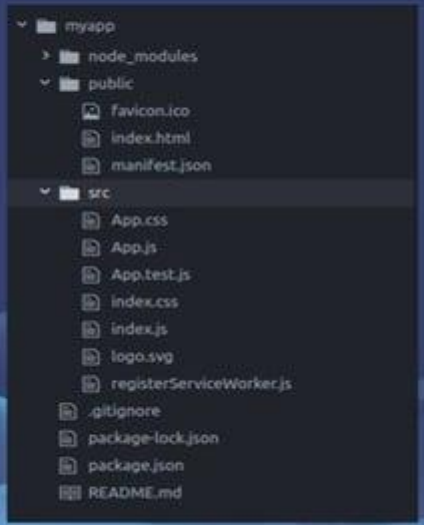


- You can run the project by typing the command cd my-app.

cd my-app
npm start



- Now you can view your app in the browser as shown in the image below :



REACT FILES AND FOLDER DETAILS

node modules -all dependencies defined by package.json

.gitignore -untracked files to not to check-in

Package.json -metadata of libraries used in project

Readme - define usage, build instructions, summary of project

Public folder -supporting files

favicon.ico (file) -icon of index file

index.html -root div container to launh

png -logo

manifest - contains metadata mobile and webapp

-Defines rules for google search

REACT FILES AND FOLDER DETAILS

src -contains components, css,tests

App.css - contains styles for our react component

App.js - basic react component replaced by root

App.test.js -A very basic file [make use of Jest]

index.js -the files to render our component

- A piece of code to be reused
- functions
- it may be more than functions as well
- search, header, footer, login components

There are 2 types of components

- Functional component
- Class component

➢ Functional components :are some of the more common components that will come across while working in React.

➢ These are simply JavaScript functions. We can create a functional component to React by writing a JavaScript function.

Syntax: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow_functions

```
const Car=()=> {
  return <h2>Hi, I am also a Car!</h2>;
}
```

- Class Component: This is the bread and butter of most modern web apps built in ReactJS.
- These components are simple classes (made up of multiple functions that add functionality to the application).

Syntax:

```
class Car extends React.Component {
  render() {
    return <h2>Hi, I am a Car!</h2>;
  }
}
```

Functional Components

- A functional component is just a plain JavaScript function that accepts props as an argument and returns a React element.
- There is no render method used in functional components.
- Also known as Stateless components as they simply accept data and display them in some form, that they are mainly responsible for rendering UI.
- React lifecycle methods cannot be used in functional components.
- Hooks can be easily used in functional components.

Class Components

- A class component requires you to extend from React. Component and create a render function which returns a React element.
- It must have the render() method returning HTML.
- Also known as Statefull components because they implement logic and state.
- React lifecycle methods can be used inside class components.
- It requires different syntax inside a class component to implement hooks.

What is JSX?

- JSX stands for JavaScript XML.
- JSX allows us to write HTML in React.
- JSX makes it easier to write and add HTML in React.

Why JSX?

- It is faster than normal JavaScript as it performs optimizations while translating to regular JavaScript.
- It makes it easier for us to create templates.
- Instead of separating the markup and logic in separated files, React uses *components* for this purpose. We will learn about components in detail in further articles.

example :

```
const element = <h1>Hello, world!</h1>;
```

-neither xml nor string

```
const name = 'Mallikarjuna';
const element = <h1>Hello, {name}</h1>;

ReactDOM.render(
  element,
  document.getElementById('root')
);
```

```
function formatName(user) {
  return user.firstName + ' ' + user.lastName;
}

const user = {
  firstName: 'Ramesh',
  lastName: 'Master'
};

const element = (
  <h1>
    Hello, {formatName(user)}!
  </h1>
);

ReactDOM.render(
  element,
  document.getElementById('root')
);
```

```
function User(){
  return <div> <h1>user</h1> </div>
}

export default User;
```

With

```
import React from 'react'

function User(){
  return React.createElement('div',null, React.createElement('h1',null,'user'))
}
```

Handling Events

- Handling events with React elements is very similar to handling events on DOM elements. There are some syntax differences:
- React events are named using camelCase, rather than lowercase.
- With JSX you pass a function as the event handler, rather than a string.

- For example, the HTML:

```
<button onclick="activateButton()">
  Activate button
</button>
```

- Is slightly different in React:

```
<button onClick={activateButton}>
  Activate button
</button>
```

- Another difference is that you cannot return false to prevent default behavior in React. You must call preventDefault explicitly.

For example, with plain HTML, to prevent the default form behavior of submitting, you can write:

```
<form onsubmit="console.log('You clicked submit.'); return false">
  <button type="submit">Submit</button>
</form>
```

In React, this could instead be:

```
function Form() {
  function handleSubmit(e) {
    e.preventDefault();
    console.log('You clicked submit.');}
  return (
    <form onSubmit={handleSubmit}>
      <button type="submit">Submit</button>
    </form> );}
```

- Event handling by using arrow Function

```
class LoggingButton extends React.Component {
 handleClick() {
   console.log('this is:', this);
 }
 render() {
   // This syntax ensures `this` is bound within handleClick
   return (
     <button onClick={() => this.handleClick()}>
       Click me
     </button>
   );
 }
}
```

DOM

- DOM stands for 'Document Object Model'. In simple terms, it is a structured representation of the HTML elements that are present in a webpage or web-app.
- DOM represents the entire UI of your application.

Virtual DOM

- React uses Virtual DOM exists which is like a lightweight copy of the actual DOM(a virtual representation of the DOM).
- So for every object that exists in the original DOM, there is an object for that in React Virtual DOM.
- It is exactly the same, but it does not have the power to directly change the layout of the document.
- Manipulating DOM is slow, but manipulating Virtual DOM is fast as nothing gets drawn on the screen.
- So each time there is a change in the state of our application, virtual DOM gets updated first instead of the real DOM.

Reconcilation

- The Virtual DOM created as snapshot of the original DOM
- The entire Virtail DOM gets updated immediately not update to orginal DOM
- The keeps two Virtual DOM i.e previous Virtual DOM and updating Virtual DOM
- Before update and get compared with current modified virtual DOM.
- The difference between previous and current VM will give which real DOM object should be updated
- It updates corresponding original DOM object
- This Real DOM change the screen

Reconcilation

```
import './App.css';
alert('i am in app')
function App() {

  function myEvent(){
    alert('welcome to my event')
    document.getElementById("test").textContent ="welcome to span"

  }
  return (
    <div className="App">
      <h1> welcome to event handling </h1>
      <span id="test">I am here</span>
      <div>about div tags</div>
      <button onClick={myEvent}>click me</button>
    </div>
  );
}

export default App;
```

- A state is an object that stores the values of properties belonging to a component, now these values can change over a period of time either while with user interactions or network changes and the state helps facilitate this functionatility.
- Every time the state changes react re-renders the component to the browser.
- The state is initialized in the constructor.
- The state can also store multiple properties.
- A method called set state (this.setState()) is used to update the value or change the value of the state object.

- Now this function performs a shallow merge between the new and the previous state. conventionally a shallow merge ensures that the previous state values are overwritten by the new state values.
- creating the state object:(The state object is initialized in the constructor)

```
import {useState} from 'react'

function App() {
  let [data,setData] = useState("Mallikarjuna")
  function updateData(){
    setData("Mallikarjuna Changed")
    alert("modified"+data)
  }
```

- The props argument passed int React Components and it is treated as HTML Attributes.
- The props stands for Properties
- For example:

```
<Student name="mallikarjuna" />
```

```
Props as function component
function Student(props){
    return (
        <h1>{props.name}</h1>
        </div>
    )
}
```

```
 props as class component
export default class Student extends React.Component{
    render(){
    return (
        <h1>{this.props.name} </h1>
        </div>
    )
    }
}
```

- Forms are really important in any website for login, signup, or whatever. It is easy to make a form but forms in React work a little differently. If you make a simple form in React it works, but it's good to add some JavaScript code to our form so that it can handle the form submission and retrieve data that the user entered. To do this we use controlled components.

Handling Forms:

- Handling forms is about how you handle the data when it changes value or gets submitted.
- In HTML, form data is usually handled by the DOM.
- In React, form data is usually handled by the components.

- When the data is handled by the components, all the data is stored in the component state.
- You can control changes by adding event handlers in the onChange attribute.
- We can use the useState Hook to keep track of each inputs value and provide a "single source of truth" for the entire application.

EXAMPLE:

```
import { useState } from "react";
import ReactDOM from 'react-dom';

function MyForm() {
  const [name, setName] = useState("");

  return (
```

```
<form>
    <label>Enter your name:
      <input
        type="text"
        value={name}
        onChange={(e) => setName(e.target.value)}
      />
    </label>
  </form>
 )
}

ReactDOM.render(<MyForm />, document.getElementById('root'));
```

OUTPUT:

Enter your name:

- We all know how important is doing the Validation of the Entered data before we do any operations using that data. Now, We will understand how to Validate Forms in React.
- we can validate forms using controlled components. But it may be time-consuming and the length of the code may increase if we need forms at many places on our website. Here comes Formik and Yup to the rescue! Formik is designed to manage forms with complex validation with ease. Formik supports synchronous and asynchronous form-level and field-level validation. Furthermore, it comes with baked-in support for schema-based form-level validation through Yup. We would also use bootstrap so that we won't waste our time on HTML and CSS.

- https://react.semantic-ui.com/

EXAMPLE:

- Let us create a form with four fields

```
import React from 'react';
import { Form, Button } from 'semantic-ui-react';

export default function FormValidation() {
  return (
    <div>
      <Form>
        <Form.Field>
          <label>First Name</label>
          <input placeholder='First Name' type="text" />
        </Form.Field>
        <Form.Field>
          <label>Last Name</label>
          <input placeholder='Last Name' type="text" />
```

EXAMPLE:

```
</Form.Field>
          <Form.Field>
            <label>Email</label>
            <input placeholder='Email' type="email" />
          </Form.Field>
          <Form.Field>
            <label>Password</label>
            <input placeholder='Password' type="password" />
          </Form.Field>
          <Button type='submit'>Submit</Button>
        </Form>
      </div>
    )
}
```

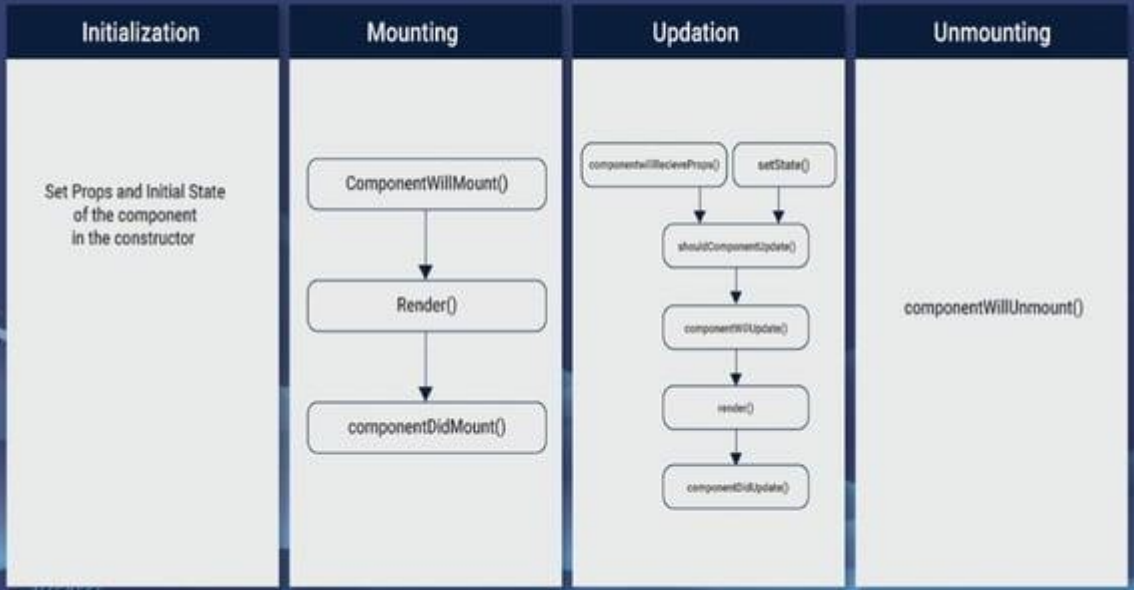
How to Add Validation to our Forms:

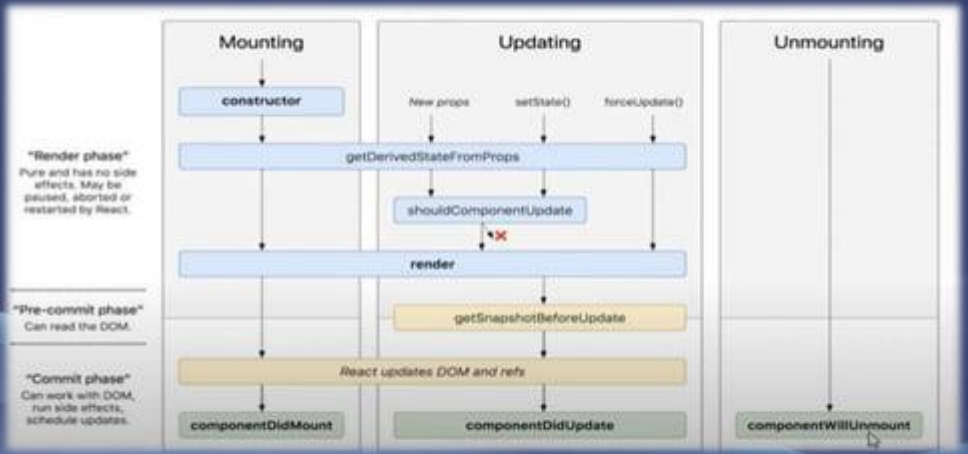
- Now, here comes the final and most awaited step. Let's add the validations.
- Let's start with the First Name field. We will use the required and maxLength properties, which are pretty self-explanatory.
- **Required** means that the field is required.
- **MaxLength** denotes the maximum length of the characters we enter.

```
<input
  placeholder='First Name'
  type="text"
  {...register("firstName", { required: true, maxLength: 10 })}
/>
```

- So, set required to true and maxLength to 10. Then if we submit the form without entering the First Name, or if the number of characters is more than 10, it will throw an error.
- Now we need to add the error message itself too. Add the following error message after the First Name form field

- Every React Component has a lifecycle of its own, lifecycle of a component can be defined as the series of methods that are invoked in different stages of the component's existence.
- React provides the developers a set of predefined functions that if present is invoked around specific events in the lifetime of the component. Developers are supposed to override the functions with desired logic to execute accordingly.
- The definition is pretty straightforward but what do we mean by different stages? A React Component can go through four stages of its life as follows.
- We have illustrated in the following diagram


Initialization
Set Props and Initial State of the component in the constructor
Mounting
ComponentWillMount()
Render()
componentDidMount()
Updation
componentWillRecievProps()
setState()
shouldComponentUpdate()
componentWillUpdate()
render()
componentDidUpdate()
Unmounting
componentWillUnmount()


Mounting
Updating
Unmounting
constructor
New props
setState()
forceUpdate()
"Render phase"
Pure and has no side effects. May be paused, aborted or restarted by React.
getDerivedStateFromProps
shouldComponentUpdate
render
"Pre-commit phase"
Can read the DOM.
getSnapshotBeforeUpdate
"Commit phase"
Can work with DOM, run side effects, schedule updates.
React updates DOM and refs
componentDidMount
componentDidUpdate
componentWillUnmount

- **Initialization:**
- This is the stage where the component is constructed with the given Props and default state. This is done in the constructor of a Component Class.
- In this phase, the developer has to define the props and initial state of the component this is generally done in the constructor of the component. The following code will describe the initialization process

Example:

```
class Clock extends React.Component{
constructor(props)
{
 //calling the constructor of
//the parent class React.component
super(props);
 //setting the initial state
this.state = { date : new Date( ) };
 }
```

- **2.Mounting:** Mounting is the phase of the component lifecycle when the initialization of the component is completed and the component is mounted on the DOM and rendered for the first time on the webpage . It means putting elements into the DOM. The mounting phase consists of two such predefined functions as described below.
- **componentWillMount() Function**: As the name clearly suggests, this function is invoked right before the component is mounted on the DOM i.e. this function gets invoked once before the render() function is executed for the first time.
- **componentDidMount() Function**: Similarly as the previous one this function is invoked right after the component is mounted on the DOM i.e. this function gets invoked once after the render() function is executed for the first time

Updating: Updating is the stage when the state of a component is updated and the application is repainted.

- React is a JS library that helps create Active web pages easily. Now active web pages are specific pages that behave according to their user.
- For example, let's take the codepen webpage, the webpage acts differently with each user. User A might write some code in C in the Light Theme while another User may write a Python code in the Dark Theme all at the same time. This dynamic behavior that partially depends upon the user itself makes the webpage an Active webpagephase.

- Now how can this be related to Updation? Updation is the phase where the states and props of a component are updated followed by some user events such as clicking, pressing a key on the keyboard, etc. The following are the descriptions of functions that are invoked at different points of Updation
- **componentWillReceiveProps() Function**: This is a Props exclusive Function and is independent of States. This function is invoked before a mounted component gets its props reassigned. The function is passed to the new set of Props which may or may not be identical to the original Props. Thus checking is a mandatory step in this regard. The following code shows a sample use-case.

Example:

```
ComponentWillReceiveProps(newProps)
{
  if(this.props !== newProps){
    console.log("new props have been assigned");
    //use this.setState( ) to re-render this page
  }
}
```

- **setState() Function**: This is not particularly a Lifecycle function and can be invoked explicitly at any instant. This function is used to update the state of a component
- **shouldComponentUpdate() Function**: By default, every state or props update re-render the page but this may not always be the desired outcome, sometimes it is desired that updating the page will not be repainted. The shouldComponentUpdate() Function fulfills the requirement by letting React know whether the component's output will be affected by the update or not. shouldComponentUpdate() is invoked before rendering an already mounted component when new props or state are being received. If returned false then the subsequent steps of rendering will not be carried out. This function can't be used in the case of forceUpdate(). The Function takes the new Props and new.

State as the arguments and returns whether to re-render or not.

- **componentWillUpdate() Function**: As the name clearly suggests, this function is invoked before the component is rerendered i.e. this function gets invoked once before the render() function is executed after the updation of State or Props.
- **componentDidUpdate() Function**: Similarly this function is invoked after the component is rerendered i.e. this function gets invoked once after the render() function is executed after the updation of State or Props.

Unmounting:

As the name suggests Unmounting is the final step of the component lifecycle where the component is removed from the page.The following function is the sole member of this function

- **componentWillUnmount() Function**: This function is invoked before the component is finally unmounted from the DOM i.e. this function gets invoked once before the component is removed from the page and this denotes the end of the lifecycle.

- Hooks allow function components to access the state and life cycle methods.
- It can be called in React function component
- Hooks are a new addition in React 16.8. They let you use state and other React features without writing a class.

Why We Need React Hooks?

- Previously, If you write a function component and notice that you need to apply some state to it all you have to do is, change that functional component into a class.

- there are many advantages of using React hooks like:
- More flexibility in reusing an existing piece of code.
- There is no need to refactor the functional component into a class component when it grows complex.
- You don't have to worry about this at all.
- No more bindings for methods
- It is simpler to distinguish logic and UI using hooks.

Basic Hooks

- useState()
- useEffect()
- useContext()

Additional Hooks

- useReducer()
- useMemo()
- useCallback()
- useImperativeHandle()
- useDebugValue()
- useRef()
- useLayoutEffect()

- The React useState Hook allows us to track state in a function component.
- State generally refers to data or properites that need to be tracking in an application.
- import { useState } from "react";
- We initialize our state by calling useState in our function component.
- useState accepts an initial state and returns two values:
  - The current state.
  - A function that updates the state.

- The React The useEffect Hook allows you to perform side effects in your components.

Some examples of side effects are: fetching data, directly updating the DOM, and timers.

- useEffect accepts two arguments. The second argument is optional.
  - useEffect(<function>, <dependency>)
- import { useState, useEffect } from "react";

- The React Context is a way to manage state globally.
- It can be used together with the useState Hook to share state between deeply nested components more easily than with useState alone.
- Challenges
  - State should be held by the highest parent component in the stack that requires access to the state.
  - To illustrate, we have many nested components. The component at the top and bottom of the stack need access to the state.
  - To do this without Context, we will need to pass the state as "props" through each nested component. This is called "prop drilling".

- Routing is a process in which a user is directed to different pages based on their action or request. ReactJS Router is mainly used for developing Single Page Web Applications. React Router is used to define multiple routes in the application. When a user types a specific URL into the browser, and if this URL path matches any 'route' inside the router file, the user will be redirected to that particular route.
- React Router is a standard library system built on top of the React and used to create routing in the React application using React Router Package. It provides the synchronous URL on the browser with data that will be displayed on the web page. It maintains the standard structure and behavior of the application and mainly used for developing single page web applications.

- React contains three different packages for routing. These are:
- react-router: It provides the core routing components and functions for the React Router applications.
- react-router-native: It is used for mobile applications.
- react-router-dom: It is used for web applications design.
- It is not possible to install react-router directly in your application. To use react routing, first, you need to install react-router-dom modules in your application. The below command is used to install react router dom.

```
$ npm install react-router-dom --save
```

Components in React Router:There are two types of router components:

- <BrowserRouter>: It is used for handling the dynamic URL.
- <HashRouter>: It is used for handling the static request

- React supports to view layer representation in each state of the application. It effectively update and render the right component with nice user interfaces.
- There are 8 different ways to styling the react components

- Inline CSS.
- Normal CSS.
- CSS in JS.
- Styled Components.
- CSS module.
- Sass & SCSS.
- Less.
- Stylable.

- Inline Styling: We can directly style an element using inline style attributes. Make sure the value of style is a JavaScript object:

```
import React from 'react'
export default class InlineComponent extends React.Component{
    render(){
        const colorStyle = {backgroundColor:'red'}
        return(
            <div>
            <h3 style={{ color: "Yellow" }}>This is a heading</h3>
            <p style={{ fontSize: "32px" }}>This is a paragraph</p>
            <p style={colorStyle  }>This is colorstyle paragraph</p>
            </div>
        )
    }

}
```

It is external CSS Styling in which separate CSS should be created with same name as component.

NormalComponent.js

```
import React from 'react'
export default class NormalComponent extends React.Component{
    render(){
        return(
            <div>
            <h1 className='headingStyles'>Normal CSS</h1>
            <h3 className='paragraphStyles'>This is a heading</h3>
         </div>
        )
    }
}
```

NormalComponent.css

```
paragraphStyles  {
    color: "Red",
    fontSize: "32px"
  };

  headingStyles {
    color: "blue",
    fontSize: "48px"
  };
```

- It is The'react-jss' integrates JSS with react app to style components. It helps to write CSS with Javascript and allows us to describe styles in a more descriptive way. It uses javascript objects to describe styles in a declarative way using 'createUseStyles' method of react-jss and incorporate those styles in functional components using className attribute.

Command to install third-party react-jss package

- npm install react-jss

```
import React from 'react'
import {createUseStyles} from 'react-jss'


export default class CSSINJS extends React.Component{

    render(){
        const styles = createUseStyles({
            paragraphStyles : {
                color: 'Red',
                fontSize: '32px'
            },
```

```
            headingStyles :{
              color: "blue",
              fontSize: "48px"
            }

});
        return(
            <div>
            <h1 className={styles.headingStyles}>CSS IN JS</h1>
            <h3 className={styles.paragraphStyles}>This is a heading</h3>
         </div>
        )
    }

}
```

CSS Modules: We can create a separate CSS module and import this module inside our component. Create a file with ".module.css" extension, styles.module.css:

```
.paragraph{
 color:"red";
 border:1px solid black;
}
```

We can import this file inside the component and use it:

```
import styles from  './styles.module.css';

class RandomComponent extends React.Component {
 render() {
   return (
     <div>
       <h3 className="heading">This is a heading</h3>
       <p className={styles.paragraph} >This is a paragraph</p>
     </div>
   );
 }
}
```

SASS and SCSS: Sass is the most stable, powerful, professional-grade CSS extension language. It's a CSS preprocessor that adds special features such as variables, nested rules, and mixins or syntactic sugar in regular CSS. The aim Is to make the coding process simpler and more efficient.

Installation:

Step 1: Before moving further, firstly we have to install node-sass , by running the following command in your project directory, with the help of terminal in your SRC folder or you can also run this command in Visual Studio Code's terminal in your project folder.

npm install node-sass

Step 2 : After the installation is complete we create a file name other.scss .

Step 3: Now include the necessary CSS effects in your CSS file.

Step 4: Now we import our file the same way we import a CSS file in React.

Step 5: In your app.js file, add this code snippet to import other.scss.

import './ other.scss';

- We are going to demonstrate how to handle errors in React Hooks. We need to create a mechanism where, if the error occurs while working with a component, user should receive an Error Component rather than throwing a run time error from the component. We will be using React Hooks in order to achieve the desired functionality.

- We will be creating a simple utility to divide 2 values, there might be a scenario, where the user might try to divide a number by 0, in this case an error should be thrown from JavaScript Code, resulting in runtime exception.

- In this case rather than terminating the program unconditionally, which is the default behavior of the application, we will display an Error Component specifying a simple runtime error. The error Component that need to be displayed in given below.

- In the below code, we can see that if the value of "hasError" is true, we are rendering the "ErrorComponent", else the normal HTML to take the user input and show the division output is displayed.

```
import React, { useState } from "react";
import "./styles.css";

export default function App() {
  var [numerator, setNumerator] = useState("");
  var [denominator, setDenominator] = useState("");
  var [executionOutput, setExecutionOutput] = useState("");
  var [hasError, setHasError] = useState(false);

  function getDivision() {
    try {
      if (denominator === "0") {
        throw Error("Division By 0");
      }
      setExecutionOutput(numerator / denominator);
    } catch {
      setHasError(true);
    }
  }
```

```
function updateValue(event) {
  if (event.target.id === "numerator") {
    setNumerator(event.target.value);
  } else {
    setDenominator(event.target.value);
  }
}

return (
  <div>
    {!hasError && (
      <section className="App">
        <div>
          <label>First Value:{" "}</label>
          <input id="numerator" type="text" value={numerator} onChange={updateValue} />
        </div>
```

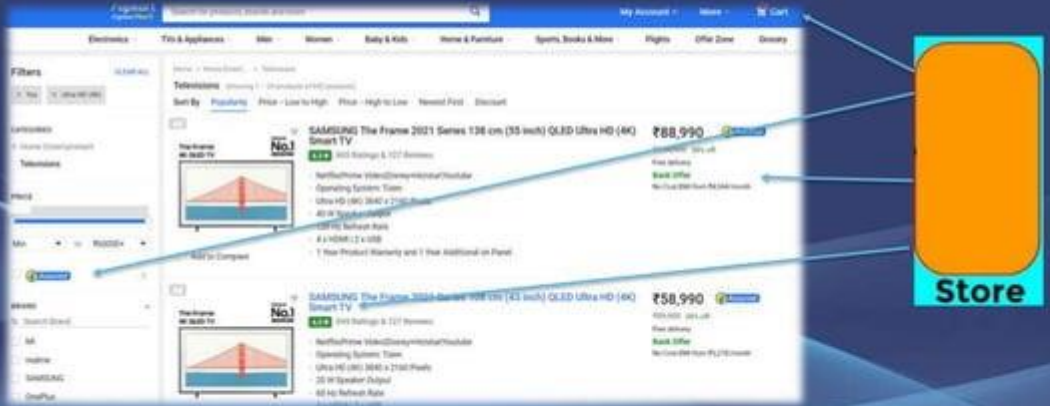
```
<div>
        <label>Second Value:{" "}</label>
        <input id="denominator" type="text" value={denominator} onChange={updateValue} />
      </div>
      <div>Output: {executionOutput}</div>
      <input type="button" onClick={getDivision} value="Divide Values" />
    </section>
  ))

  {hasError && <ErrorComponent></ErrorComponent>}
 </div>
 );
}

function ErrorComponent() {
 return <h1>Division by 0 Error</h1>
}
```

- State variable for "numerator", "denominator" and "executionOutput"
- State Variable to track error Condition "hasError"
- As soon as the error condition arrives, "hasError" marked as true
- if "hasError" is true, render "ErrorComponent"
- If "hasError" is false, render the normal component to take user input

- Flux : Flux is an architectural pattern introduced by "Facebook" to work with React. It is a slight modification of the observer-observable pattern and it is not a library or a framework. The main feature in Flux is the concept of uni-directional data flow..

- Redux : Redux is a library inspired by Flux and can be considered as an implementation of Flux. Redux makes easy to handle the state of the application and manage to display data on user actions. It is a very powerful library but also very lightweight.Redux is created by Dan Abramov and Andrew Clark, both were React core team at Facebook.

- Redux is container where you can store your whole application data
- It is also considered to state manegment
- It doesn't belong to component state



Diffenrence b/w Redux and Flux

Redux	Flux
It is a library for managing states that follow the principles of the flux architecture	Flux pattern is constructed based on four segments organized in a uni-directional manner.Action,Dispatcher,Store,View
In Redux, the dispatcher is replaced with the reducers	In Flux,logic of what needed to be executed based on the action is written in the store
Redux is maintaining a large single immutable store. On every action, it takes the old store, makes a copy, applies the new changes and makes it as the new store.	
The components are Store, Actions, and Reducer	

What are the three principles that Redux follows ?

- Redux is a Single Source of Truth
- The State is Read-only State
- The Modifications are Done with Pure Functions

1. Redux is a Single Source of Truth

The global state of an app is stored within an object tree in a single store

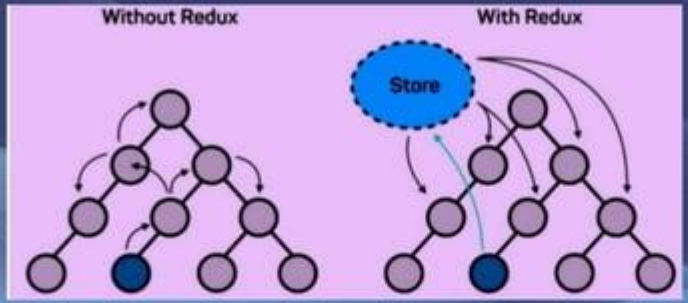
2. The State is Read-only State

There is only one way for changing the state-emit an action or an object that describes what happened

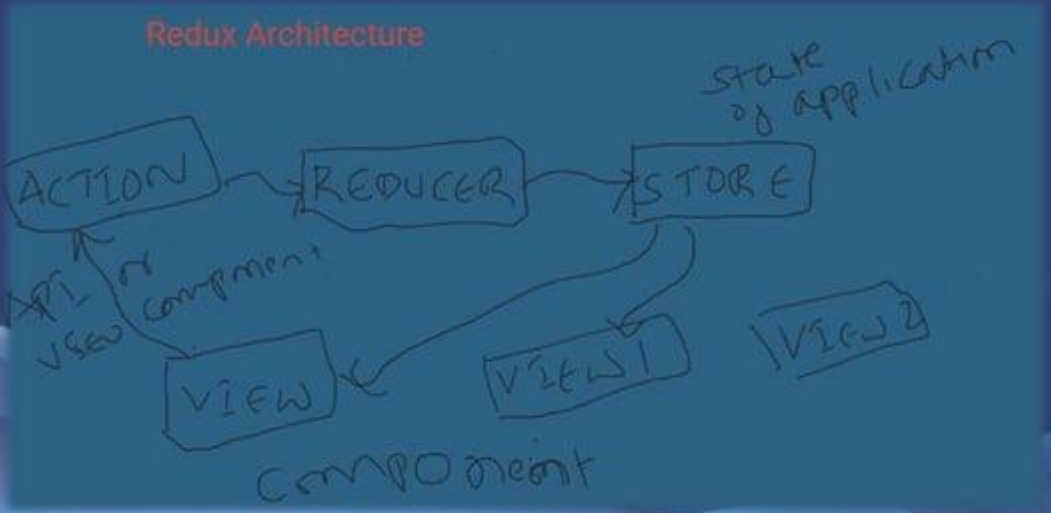
The application state cannot be mutated. All changes to state must come through the use of pure functions called "reducers" which take a previous state and a new action and determine what the resulting state should be.

- The Modifications are Done with Pure Functions

- The reducers are merely pure functions, which take the previous state as well as action
- and move it to the next state
- To specify how the state tree is transformed by actions, you write pure reducers.
- #2 overlaps with #3: reducers must be pure functions,



Redux Architecture



- Action

An object setting up information about our data moving into state.

- Dispatch

Functions that act as a bridge between our actions and reducers, sending the information over to our application.

- Reducer

A function that receives data information from our action in the form of a type and payload and modifies the state based on additional provided data.

- Provider

We can then use the provider to wrap our store around our application and by doing so, pass and render our state. We can then use connect with a component and enable it to receive store state from the provider.

- Containers

Containers are the primary components and the entry point from which we call child or generic components. Basically, container components are called smart components because each container component is connected to Redux and consumes the global data coming from the store.

Using Redux for state management - overview

Redux contains - Store, Reducers, Actions, Providers

Action

these are objects that should have two properties, one describing the type of action, and one describing what should be changed in the app state

An object setting up information about our data moving into state.

Reducer

A function that receives data information from our action in the form of a type and payload and modifies the state based on additional provided data.these are functions that implement the behavior of the actions. They change the state of the app, based on the action description and the state change description.

Store

Once we have our actions and reducers set up, everything is maintained in our store. The store is where we can set up our initial state, combine our reducers, apply middleware, and hold our information.

it brings the actions and reducers together, holding and changing the state for the whole app — there is only one store.

Sample example

https://react-redux.js.org/tutorials/quick-start

https://www.youtube.com/watch?v=CtWAi5LYKCY

Axios, which is a popular library is mainly used to send asynchronous HTTP requests to REST endpoints. This library is very useful to perform CRUD operations.

1.This popular library is used to communicate with the backend. Axios supports the Promise API, native to JS ES6.

2.Using Axios we make API requests in our application. Once the request is made we get the data in Return, and then we use this data in our project.

3.This library is very popular among developers. You can check on GitHub and you will find 78k stars on it.

npm install axios

Why AXIOS in React?

It has good defaults to work with JSON data. Unlike alternatives such as the Fetch API, you often don't need to set your headers. Or perform tedious tasks like converting your request body to a JSON string.

- Axios has function names that match any HTTP methods. To perform a GET request, you use the .get() method.
- Axios does more with less code. Unlike the Fetch API, you only need one .then() callback to access your requested JSON data.
- Axios has better error handling. Axios throws 400 and 500 range errors for you. Unlike the Fetch API, where you have to check the status code and throw the error yourself.
- Axios can be used on the server as well as the client. If you are writing a Node.js application, be aware that Axios can also be used in an environment separate from the browser.

https://jsonplaceholder.typicode.com/

Free fake API for testing and prototyping.

All HTTP methods are supported. You can use http or https for your requests.

GET	/posts
GET	/posts/1
GET	/posts/1/comments
GET	/comments?postId=1
POST	/posts
PUT	/posts/1
PATCH	/posts/1
DELETE	/posts/1

Best Practices in React App Development

- Decompose into Small Components
- Use Functional or Class Components based on Requirement
- Use Functional Components with Hooks
- Appropriate Naming and Destructuring Props
- Use propTypes for Type Checking and Preventing Errors
- Naming Conventions
- A component name should always be in a Pascal case like 'SelectButton', 'Dashboard' etc. Using Pascal case for components differentiate it from default JSX element tags.
- Methods/functions defined inside components should be in Camel case like 'getApplicationData()', 'showText()' etc.

- Project Structure
- The architecture focuses on reusable components of the react developer architecture so that the design pattern can be shared among multiple internal projects

- Children Props
- Sometimes it is required to render method the content of one component inside another component. So we can pass functions as children props which get called components render function.

- React performance optimization techniques

- 1. Keeping component state local where necessary
- To ensure re-rendering a component only happens when necessary, we can extract the part of code that cares about the component state, making it local to that part of the code.

- 2. Memoizing React components to prevent unnecessary re-renders
- Memoization is an optimization strategy that caches a component-rendered operation, saves the result in memory, and returns the cached result for the same input.

- 3. Code-splitting in React using dynamic import()
- With code-splitting, React allows us to split a large bundle file into multiple chunks using dynamic import() followed by lazy loading these chunks on-demand using the React.lazy. This strategy greatly improves the page performance of a complex React application

- 4. Windowing or list virtualization in React applications
- Imagine we have an application where we render several rows of items on a page. Whether or not any of the items display in the browser viewport, they render in the DOM and may affect the performance of our application.
- With the concept of windowing, we can render to the DOM only the visible portion to the user. Then, when scrolling, the remaining list items render while replacing the items that exit the viewport. This technique can greatly improve the rendering performance of a large list.
- Both react-window and react-virtualized are two popular windowing libraries that can implement this concept.

- 5. Lazy loading images in React
- Similar to the concept of windowing mentioned above, lazy loading images prevents the creation of unnecessary DOM nodes, boosting the performance of our React application.
- react-lazyload and react-lazy-load-image-component are popular lazy loading libraries that can be used in React projects.

4. Using Immutable Data Structures

- Data immutability is a practice that revolves around a strict unidirectional data flow.

- 2. Function/Stateless Components and React.PureComponent
- function components and PureComponent provide two different ways of optimizing React apps at the component level.

- 3.Avoid Inline Function Definition in the Render Function
- Since functions are objects in JavaScript ({} !== {}), the inline function will always fail the prop diff when React does a diff check. Also, an arrow function will create a new instance of the function on each render if it's used in a JSX property. This might create a lot of work for the garbage collector.

THANK YOU